CO1 - AT - 1

Short Answer Test.

Name: R.S. Kavinadhithya
Reg.No: 192511365
Course: Operating System
Code: CSA0408
Date: 08/10/2026

22/25

1. Scenario:

A Modern Operating System supports multiple users and processes with shared resources like CPU, memory and I/O devices:

a) Type of operating System:

* Multi user OS - Allow Multiple users to use the computer system simultaneously
* Multitasking OS - Allow Multiple processes/ Program to run apparently at the same time.
* Multiprogramming OS - keeps Multiple program in memory and shares the CPU among them
* Time - Sharing OS - gives each process user a small CPU time slice for quick response.

b) OS structure showing Resource Management Users:

↓

Application Program

↓

System calls.

↓

operating System

* Process Management
* Memory Management
* File Management
* I/O Management
* Security.

↓

Hardware Resources:

* CPU
* Main Memory
* Disk / Files
* I/O Devices.

c) System calls to create and Execute New Process:

Parent Process

↓

fork()

↓

child Process Created

↓

exec()

↓

New Program Executed

↓

Wait

↓

Parent waits for child

↓

exit()

Scenario:

a) Categories of system calls for File and device Management:

→ File Management system calls Used to Manage File:

* open() - open a file
* read() - reads data
* write() - write data
* close() - close a file
* lseek() - change file posit

→ Device Management system calls: Used to control I/O devices:

* open() - opens a device
* read() - Reads from device
* write() - writes to device
* ioctl() - controls devices operation
* close() - closes devices.

b) Layered structure for file operation:

User Program
↓
System Management
↓
File Management
↓
File System Layer
↓

Device Driver

↓

I/O Management

↓

Hardware

(Disk / Device)

c) C-Based System call sequence to open. Read and close a File:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
  int fd;
  char buffer[100];
  fd = open("data.text", O_RDONLY);
  read(fd, buffer, sizeof(buffer));
  close(fd);
  return 0;
}
```

Sequence:

open()

↓

read()

↓

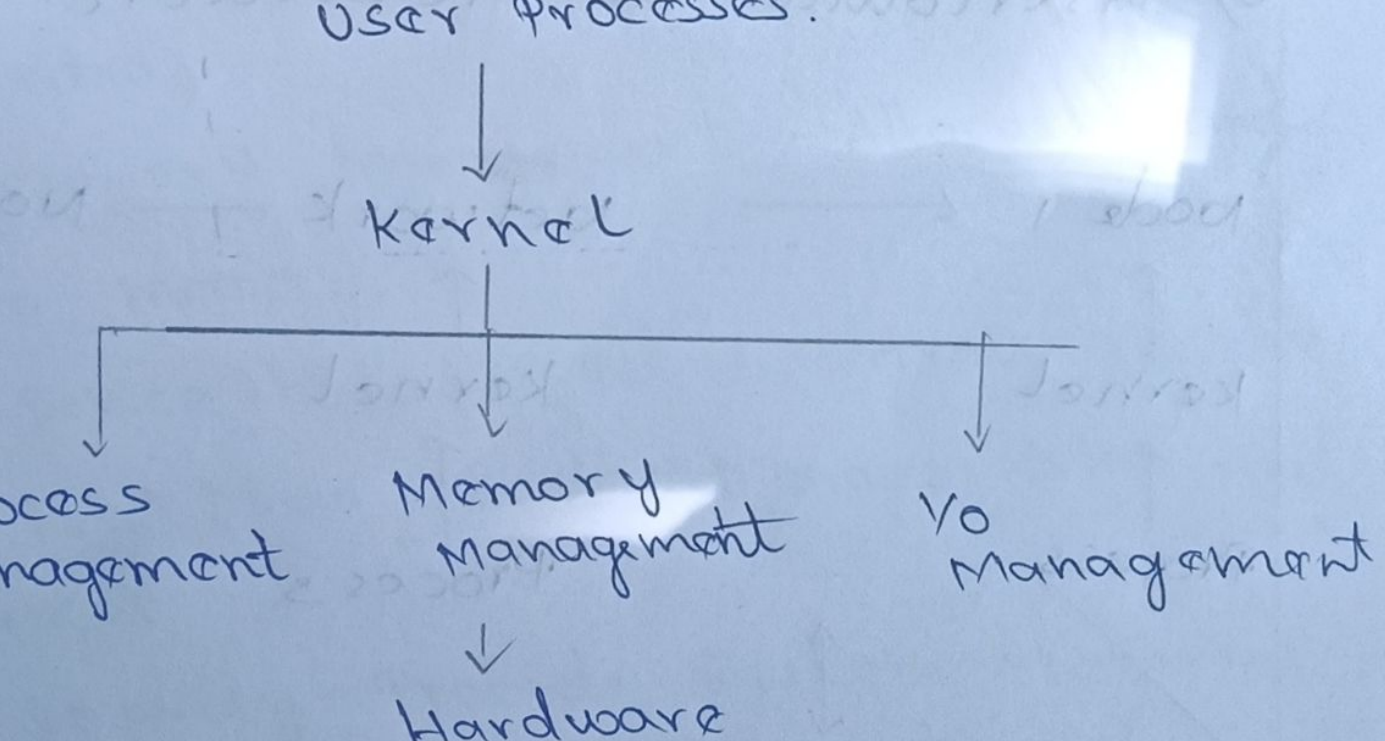
close()

a) Type of OS:

Real Time operating System [RTOS] is suitable for industrial machinery.

* Uyires fast response
* Provides predictable timing
* Handles critical operations within a deadline.
* Reliable for controlling Machines.

b) Interation Diagram:



System calls for Process Synchronization

* Semaphore - Controls access to shared resources.
* wait () - Makes a process wait
* signal () - Releases / wakes a waiting Process.
* Mutex Lock/ unlock - Provides Mutual.

4. a) Distributed os vs centralized os

Type : Distributed operating system.

Distributed os centralized os

Uses Multiple connected computer Uses one main computer.

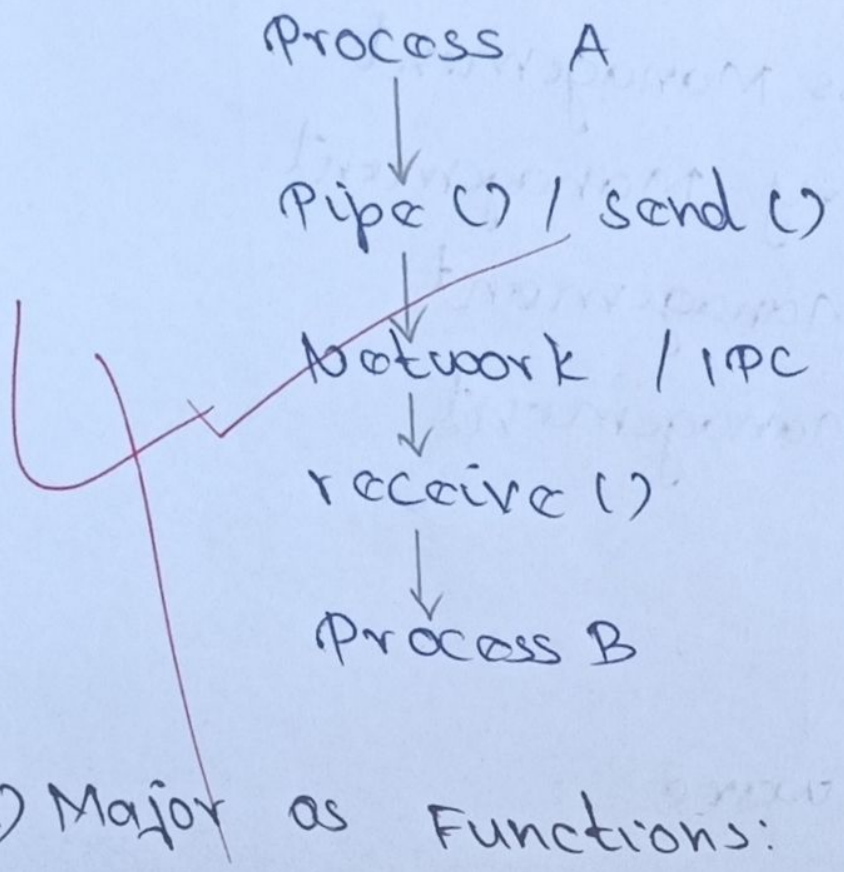
Resources are shared over network Resources are managed centrally.

Provides better resource sharing limited central resources.

b) Distributed os Architecture:

Common calls : Send(), receive(), pipe(), shmget(), shmat().

c) IPC System call Sequence:



5. a) Major os Functions:

* Process scheduling - Allocates CPU to Processes.
* Memory Management - Allocates and frees memory
* Device Management - controls I/O devices
* File Management - Manages files and storage.
* Security - Protects system resources.

b) os components and Interaction:

User / Application
↓
System calls.
↓

OS / kernel:

* Process Management
* Memory Management
* File Management
* I/O Management

↓

Hardware

System Call Example:

Memory : Mmap(), brk()

Device : open(), read(), write(), ioctl().